All five Python scripts are aligned with your problem statements and use modern Qiskit syntax (qiskit_aer and dynamic if_test circuits). However, there is an important nuance in **Assignment 3** that you should understand before an exam or code review:

Assignment	Problem Statement	Alignment Verdict	Key Implementation Detail
1. assign1.py	16-Qubit Random Number Generator	**100% Aligned**	Uses Hadamard gates on 16 qubits with shots=1 to extract a true random 16-bit binary string.
2. assign2.py	Tackle Noise with Error Correction	**100% Aligned**	Implements the **3-Qubit Bit-Flip Repetition Code** using 2 ancilla qubits and dynamic syndrome decoding.
3. assign3.py	Tarrataca's QPS with 3-Puzzle	**Conceptual / Toy Alignment**	A full 3-puzzle board requires at least 8 qubits. This script implements a **3-qubit Grover's Search algorithm** to demonstrate the *rule-matching inference engine* at the heart of Tarrataca's Quantum Production System.
4. assign4.py	Quantum Teleportation Algorithm	**100% Aligned**	Teleports a rotated state ($R_y(\pi/3)$) from Alice to Bob and proves success by applying the

			inverse rotation ($R_y(-\pi/3)$).
5. assign5.py	5-Qubit Quantum Fourier Transform	**100% Aligned**	Implements the 5-qubit QFT circuit using Hadamard gates, iterative Controlled-Phase (CP) rotations, and final $SWAP$ gates.

Part 1: Quantum AI Foundations (Your Prerequisite Toolkit)

Before reading the code line-by-line, you need six core concepts that power all five scripts:

1. The Qubit and Superposition

A classical bit is strictly 0 or 1. A **qubit** exists in a mathematical state represented in **Dirac (bra-ket) notation** as a linear combination (superposition) of two basis states, $\vert{}0\rangle$ and $\vert{}1\rangle$:

$$\vert{}\psi\rangle = \alpha\vert{}0\rangle + \beta\vert{}1\rangle$$

Here, α and β are complex numbers called **probability amplitudes**.

- So long as you do not look at the qubit, it holds both possibilities simultaneously.
- When you **measure** the qubit, the superposition collapses: it becomes classical 0 with probability $\vert{}\alpha\vert{}^2$ and classical 1 with probability $\vert{}\beta\vert{}^2$, where $\vert{}\alpha\vert{}^2 + \vert{}\beta\vert{}^2 = 1$.

2. Phase (The Hidden Dimension)

If a qubit is in the state $\frac{1}{\sqrt{2}}(\vert{}0\rangle + \vert{}1\rangle)$ versus $\frac{1}{\sqrt{2}}(\vert{}0\rangle - \vert{}1\rangle)$, both have a 50% chance of measuring 0 and a 50% chance of measuring 1 (since $\vert{}-\frac{1}{\sqrt{2}}\vert{}^2 = \frac{1}{2}$). That minus sign is a **relative phase**. While you cannot see phase by measuring right away, quantum gates cause phases to collide—canceling out wrong answers (**destructive interference**) and amplifying right answers (**constructive interference**).

3. Essential Quantum Gates Used in Your Code

Quantum gates are matrices that rotate or flip qubit states:

- X **Gate (Pauli-X / Quantum NOT):** Flips $\vert{}0\rangle \leftrightarrow \vert{}1\rangle$. Used in your code to simulate bit-flip noise or apply bit corrections.
- Z **Gate (Pauli-Z / Phase Flip):** Leaves $\vert{}0\rangle$ alone, but flips the sign of $\vert{}1\rangle \rightarrow -\vert{}1\rangle$. Used in teleportation recovery.
- H **Gate (Hadamard):** The superposition creator. It turns $\vert{}0\rangle \rightarrow \frac{\vert{}0\rangle + \vert{}1\rangle}{\sqrt{2}}$ (called $\vert{}+\rangle$) and

$\vert{}1\rangle \rightarrow \frac{\vert{}0\rangle - \vert{}1\rangle}{\sqrt{2}}$ (called $\vert{}-\rangle$). Crucially, applying H a second time undoes the superposition ($H \cdot H = I$).

- $R_y(\theta)$ **Gate:** Rotates a qubit around the Y-axis of the Bloch sphere by angle θ, creating an unequal superposition: $\cos(\theta/2)\vert{}0\rangle + \sin(\theta/2)\vert{}1\rangle$.
- CX **Gate (CNOT / Controlled-NOT):** A 2-qubit gate with a **control** qubit and a **target** qubit. If the control qubit is $\vert{}1\rangle$, it flips (X) the target qubit; if the control is $\vert{}0\rangle$, it does nothing. When the control qubit is in superposition, CX creates **Entanglement**.
- CCX **Gate (Toffoli / Controlled-Controlled-NOT):** A 3-qubit gate (like a quantum AND gate). It flips the third qubit *only* if both the first and second qubits are $\vert{}1\rangle$.
- $CP(\theta)$ **Gate (Controlled-Phase):** Adds a phase rotation $e^{i\theta}$ to the target qubit only if both the control and target qubits are $\vert{}1\rangle$.
- $SWAP$ **Gate:** Exchanges the quantum states of two qubits.

4. Qiskit's "Little-Endian" Ordering (Crucial!)

In standard reading, we write binary numbers with the most significant bit on the left ($q_0 q_1 q_2$).

Qiskit reverses this (Little-Endian ordering): qubit 0 is placed on the **far right** of the bitstring ($q_2 q_1 q_0$). Keep this in mind when reading measurement strings and syndrome integers in assign2.py.

Part 2: Deep-Dive into Each Assignment

Assignment 1: 16-Qubit Quantum Random Number Generator (assign1.py)

The Theory

Classical computers cannot generate true random numbers; they use Pseudo-Random Number Generators (PRNGs) driven by deterministic mathematical formulas and a starting "seed." Quantum mechanics is intrinsically probabilistic. By placing 16 qubits into an equal superposition using Hadamard gates, the system enters an equal superposition of all $2^{16} = 65,536$ possible binary numbers:

$$\vert{}\Psi\rangle = H^{\otimes 16}\vert{}0\rangle^{\otimes 16} = \frac{1}{\sqrt{2^{16}}} \sum_{x=0}^{65535} \vert{}x\rangle$$

Measuring this state forces nature to randomly collapse into one of those 65,536 states with an exact probability of $\frac{1}{65536}$ each.

Code Walkthrough

- qc = QuantumCircuit(16, 16): Allocates 16 qubits (initialized to $\vert{}0\rangle$) and 16 classical bits to store the measurement output.
- for i in range(16): qc.h(i): Applies a Hadamard gate to every qubit, putting each into the $\frac{\vert{}0\rangle + \vert{}1\rangle}{\sqrt{2}}$ state.
- qc.measure(range(16), range(16)): Measures qubit i and writes the collapsed classical bit

(0 or 1) into classical bit i.

- compiled_circuit = transpile(qc, simulator): Optimizes and translates the circuit into instructions the AerSimulator backend understands.
- job = simulator.run(compiled_circuit, shots=1): Runs the circuit **only once** (shots=1). If you ran 1,000 shots, you would get a histogram of 1,000 different random numbers; using shots=1 guarantees a single collapse.
- list(result.keys())[0]: Extracts the single 16-character binary string (e.g., '1011001011010100') from Qiskit's results dictionary.

Assignment 2: Tackling Noise with Error Correction (assign2.py)

The Theory

Real quantum computers suffer from **decoherence** and **noise**—stray magnetic fields or thermal fluctuations can accidentally flip a qubit from $|0\rangle$ to $|1\rangle$ (a **bit-flip error**). In classical computing, we fix bit flips using repetition: copy the bit 3 times (0 → 000), and if one flips (010), take a majority vote (0). In quantum computing, two massive roadblocks prevent simple copying and voting:

1. **The No-Cloning Theorem:** Physics forbids making an exact independent copy of an unknown quantum state $|\psi\rangle$.
2. **Measurement Collapse:** If you measure the qubits to take a majority vote, you destroy their quantum superposition.

The Solution (3-Qubit Bit-Flip Code):

1. **Entanglement Encoding:** Instead of copying, we *entangle* our data qubit with two helper data qubits using CX gates: $\alpha|0\rangle + \beta|1\rangle \rightarrow \alpha|000\rangle + \beta|111\rangle$.
2. **Syndrome Extraction (Parity Checks):** We introduce 2 extra **Ancilla (scratchpad) qubits**. Instead of measuring the data qubits directly, we use CX gates to ask the ancillas **parity questions** (*"Are data qubits 0 and 1 the same or different?"*). Measuring the ancillas gives us a 2-bit clue called the **Error Syndrome** that tells us *which* qubit flipped without revealing whether the data qubits are 0 or 1!

Code Walkthrough

- **Registers:** Creates 3 data qubits (q_data), 2 ancilla qubits (q_ancilla), 2 syndrome classical bits (c_syndrome), and 3 output classical bits (c_out).
- **Step 1: Encoding (qc.cx(q_data[0], q_data[1]) & qc.cx(q_data[0], q_data[2])):** Spreads the logical state of q_data[0] across all three data qubits. Since q_data[0] starts at $\vert{}0\rangle$, the encoded logical state is $\vert{}000\rangle$.
- **Step 2: Injecting Artificial Noise (qc.x(q_data[1])):** Simulates a hardware bit-flip error on the middle qubit (q_data[1]), corrupting our state from $\vert{}000\rangle$ to $\vert{}010\rangle$.
- **Step 3: Parity Check 1 (qc.cx(q_data[0], q_ancilla[0]) & qc.cx(q_data[1], q_ancilla[0])):** Checks if q_data[0] and q_data[1] match. If both are 0 or both are 1, q_ancilla[0] gets flipped zero or two times, remaining 0. If one of them flipped, q_ancilla[0] gets flipped once, becoming 1.
- **Step 4: Parity Check 2 (qc.cx(q_data[1], q_ancilla[1]) & qc.cx(q_data[2],**

q_ancilla[1])): Checks if q_data[1] and q_data[2] match.

- **Step 5: Syndrome Measurement & Decoding (with qc.if_test(...)):** Measures q_ancilla into c_syndrome. Because Qiskit uses **Little-Endian** order ([q_ancilla[1], q_ancilla[0]]), here is how the binary syndrome maps to the integer in if_test:

Corrupted Qubit	q_ancilla[1] (b1b_1b1)	q_ancilla[0] (b0b_0b0)	Binary (b1b0b_1b_0b1b0)	Decimal Integer in if_test	Recovery Action
None	0	0	00	0	Do nothing
q_data[0]	0	1	01	**1**	qc.x(q_data[0])
q_data[1]	1	1	11	**3**	qc.x(q_data[1])
q_data[2]	1	0	10	**2**	qc.x(q_data[2])

- **Output:** Since q_data[1] was flipped, c_syndrome measures 11 (integer 3), triggering qc.x(q_data[1]) to repair the state back to 000. The printed output will be {'000 11': 1000}—meaning all 1,000 runs had syndrome 11 and were successfully repaired to 000!

Assignment 3: Tarrataca's Quantum Production System (assign3.py)

The Theory: Classical vs. Quantum Production Systems

In classical Artificial Intelligence, a **Production System** is a cognitive rule-based architecture used for automated problem solving (like the **3-Puzzle**, where 3 numbered tiles and 1 blank space slide on a 2×2 grid). It has three components:

1. **Working Memory:** Holds the current board state.
2. **Production Memory:** A database of IF (condition) THEN (action) rules (e.g., *"IF blank is at top-left, THEN slide tile from top-right"*).
3. **Inference Engine:** Matches the Working Memory against the Production Memory to find valid rules, resolves conflicts, and fires the rule.

In classical AI, searching through rule sequences takes exponential time. Researchers **Luís Tarrataca and Andreas Wichert** proposed the **Quantum Production System (QPS)**, which converts the Inference Engine into a quantum circuit using **Grover's Search Algorithm**:

1. Put all candidate production rules / board states into an equal superposition.
2. Use a **Quantum Oracle** to evaluate the IF condition of all rules in parallel, flipping the phase (adding a - sign) of the valid rule(s).
3. Use a **Grover Diffusion Operator** (inversion about the mean) to convert that hidden - phase into a large measurement probability.

Note on assign3.py: Encoding a complete 2×2 3-puzzle board and its transition rules requires

at least 8 to 12 qubits. Your script implements a **3-qubit abstraction of the QPS Inference Engine** using Grover's algorithm to search $2^3 = 8$ candidate states (000 to 111) and match the rule corresponding to state |111>.

Code Walkthrough

- **1. Initialize Working Memory (qps_circuit.h(range(3))):** Puts all 3 qubits into an equal superposition of all 8 possible states ($\vert{}000\rangle$ through $\vert{}111\rangle$), each with amplitude $+\frac{1}{\sqrt{8}}$ (probability $\frac{1}{8} = 12.5\%$).
- **2. The Rule-Matching Oracle (build_qps_puzzle_oracle()):**
  - Contains qc.h(2), qc.ccx(0, 1, 2), and qc.h(2).
  - **Mathematical identity ($H \cdot X \cdot H = Z$):** Sandwiching the target of a Toffoli (ccx) gate between two Hadamard gates turns a Controlled-Controlled-NOT into a **Controlled-Controlled-Z (CCZ) gate**.
  - This gate checks if Qubit 0, Qubit 1, AND Qubit 2 are all 1. If so, it flips the phase of $\vert{}111\rangle$ from $+\frac{1}{\sqrt{8}}\vert{}111\rangle$ to $-\frac{1}{\sqrt{8}}\vert{}111\rangle$ while leaving the other 7 states untouched.
- **3. The Diffusion Operator (diffuser):**
  - At this point, $\vert{}111\rangle$ is marked with a - sign, but if we measure now, $\vert{}-\frac{1}{\sqrt{8}}\vert{}^2$ is still just $\frac{1}{8}$ (12.5%).
  - The sequence h(range(3)) → x(range(3)) → h(2) + ccx(0,1,2) + h(2) → x(range(3)) → h(range(3)) performs **Amplitude Amplification** (reflecting all amplitudes across their average value).
  - Because $\vert{}111\rangle$ had a negative amplitude, reflecting across the mean boosts its probability from **12.5% to 78.125%** ($\frac{25}{32}$) in a single step!
- **Output:** Out of 1,024 shots, you will see '111' measured roughly **800 times**, while the other 7 invalid rules share the remaining ~224 shots.

Assignment 4: Quantum Teleportation Algorithm (assign4.py)

The Theory

Quantum Teleportation transfers an unknown quantum state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ from **Alice** (q0) to **Bob** (q2), even if they are miles apart, without physically sending the qubit through space.

- **How it works:** Alice and Bob pre-share an entangled **Bell Pair** (q1 held by Alice, q2 held by Bob) in the state $\vert{}\Phi^+\rangle = \frac{\vert{}00\rangle + \vert{}11\rangle}{\sqrt{2}}$. Alice entangles her message qubit (q0) with her half of the Bell pair (q1) and measures both.
- Measuring q0 and q1 collapses Alice's qubits (obeying the No-Cloning Theorem, as Alice's original copy is destroyed) and projects Bob's qubit (q2) into one of four states: $\vert{}\psi\rangle$, $X\vert{}\psi\rangle$, $Z\vert{}\psi\rangle$, or $XZ\vert{}\psi\rangle$.
- Alice sends her 2 classical measurement bits (crz and crx) to Bob over a normal communication channel (obeying the speed of light), telling Bob which corrective gate (X and/or Z) to apply to unlock $\vert{}\psi\rangle$.

Code Walkthrough

- **Step 1: Prepare Message State (qc.ry(np.pi/3, 0)):** Rotates Alice's message qubit q0 by $\theta = \pi/3$ (60°), creating the state $\vert{}\psi\rangle = \cos(\pi/6)\vert{}0\rangle + \sin(\pi/6)\vert{}1\rangle = \frac{\sqrt{3}}{2}\vert{}0\rangle + \frac{1}{2}\vert{}1\rangle$.
- **Step 2: Create Shared Entanglement (qc.h(1) & qc.cx(1, 2)):** Creates the entangled Bell pair between Alice's q1 and Bob's q2.
- **Step 3: Bell-Basis Measurement (qc.cx(0, 1), qc.h(0), measure):** Alice entangles q0 and q1 and measures them into classical registers crz and crx.
- **Step 4: Bob's Conditional Recovery (with qc.if_test(...)):**
  - If Alice's crx bit is 1, Bob applies qc.x(2) to fix a bit flip.
  - If Alice's crz bit is 1, Bob applies qc.z(2) to fix a phase flip.
  - At this exact line, Bob's qubit q2 is guaranteed to be in the exact state $R_y(\pi/3)\vert{}0\rangle$!
- **Step 5: The Verification Trick (qc.ry(-np.pi/3, 2)):**
  - How do we *prove* Bob's qubit has the exact angle $\pi/3$ (including phase)? We apply the **exact inverse rotation** $R_y(-\pi/3)$ to Bob's qubit q2 right before measuring it!
  - Since $R_y(-\pi/3) \cdot R_y(+\pi/3) = I$ (they cancel out completely), Bob's qubit is rotated straight back to pure $\vert{}0\rangle$.
- **Output:** In result_cr (the left-most bit in the printed string), you will see 0 **100% of the time** across all cuatro combinations of Alice's random crx and crz bits ('0 0 0', '0 0 1', '0 1 0', '0 1 1'), proving the state was teleported intact.

Assignment 5: 5-Qubit Quantum Fourier Transform (assign5.py)

The Theory

The **Discrete Fourier Transform (DFT)** is one of the most important algorithms in classical engineering, converting signals from the time domain into the frequency domain. Taking the classical Fast Fourier Transform (FFT) on $N = 2^n$ data points takes $O(n2^n)$ operations. The **Quantum Fourier Transform (QFT)** performs the exact same mathematical transformation on the amplitudes of an n-qubit quantum state using only $O(n^2)$ **gates**—an exponential speedup that powers Shor's factoring algorithm and Quantum Phase Estimation.

Instead of encoding numbers in the **computational basis** (where $|x\rangle$ is a string of 0s and 1s), QFT translates that number into the **Fourier basis** (where the value of x is encoded into the *angles of rotation* around the Z-axis of each qubit in superposition).

Code Walkthrough

- **The Nested Loops (for i in range(5): and for j in range(i + 1, 5):):**
  - For each qubit i, qc.h(i) first puts it into superposition (handling its own binary digit).
  - Next, the inner loop j iterates over all subsequent qubits and applies a **Controlled-Phase (qc.cp(angle, j, i))** gate.
  - **The Angle Formula (angle = np.pi / (2 ** (j - i))):** Each subsequent qubit j contributes a progressively smaller fractional phase rotation based on its distance $(j - i)$ from qubit i:
    - When $j - i = 1$: $\theta = \frac{\pi}{2^1} = \frac{\pi}{2}$ (90°).

- When $j - i = 2$: $\theta = \frac{\pi}{2^2} = \frac{\pi}{4}$ (45°).
- When $j - i = 3$: $\theta = \frac{\pi}{2^3} = \frac{\pi}{8}$ (22.5°).
- When $j - i = 4$: $\theta = \frac{\pi}{2^4} = \frac{\pi}{16}$ (11.25°).

- **Why qc.swap(0, 4) and qc.swap(1, 3) are at the end:**
  - In the loop above, Qubit 0 receives controlled-phase rotations from all 4 other qubits (1, 2, 3, 4), while Qubit 4 receives *zero* controlled-phase rotations (only a single Hadamard gate qc.h(4)).
  - Mathematically, the QFT circuit outputs the qubits in **reversed order of significance** (the most significant Fourier bit ends up where the least significant computational bit started).
  - Swapping outer pair (0, 4) and inner pair (1, 3) (while middle qubit 2 stays in place) reverses the 5-qubit register back into the proper order.
- **Circuit Depth (qft_circuit.depth()):** Prints the length of the longest sequential path of gates through the circuit (excluding barriers), showing how compact the $O(n^2)$ gate scaling is for $n = 5$.

Would you like to explore how to modify assign3.py if an instructor asks you to encode actual 3-puzzle board moves, or dive deeper into the matrix math of any of these circuits?